🔷 MASTER TIME COMPLEXITY CHEAT SHEET (CCEE – CDAC)

Rule of thumb for CCEE:
Always focus on **Worst Case Time Complexity** unless explicitly stated.

---

1. BASIC OPERATIONS

Operation	Time Complexity
Variable assignment	O(1)
Arithmetic operation (+, −, ×, ÷)	O(1)
Comparison	O(1)
Return statement	O(1)
Function call (no recursion)	O(1)

---

2. ARRAY OPERATIONS

Operation	Best	Average	Worst
Access by index	O(1)	O(1)	O(1)
Search (unsorted)	O(1)	O(n)	O(n)
Search (sorted, linear)	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)
Insert (end)	O(1)	O(1)	O(1)
Insert (beginning/middle)	O(n)	O(n)	O(n)
Delete (end)	O(1)	O(1)	O(1)
Delete (beginning/middle)	O(n)	O(n)	O(n)

---

3. LINKED LIST OPERATIONS

Operation	Time Complexity
Access by index	O(n)
Search	O(n)
Insert (head)	O(1)

Insert (tail, known pointer)	O(1)
Insert (tail, unknown pointer)	O(n)
Delete (head)	O(1)
Delete (tail)	O(n)

---

4. STACK & QUEUE

Operation	Time Complexity
Push	O(1)
Pop	O(1)
Peek / Front	O(1)
IsEmpty / IsFull	O(1)

---

5. SEARCHING ALGORITHMS

Algorithm	Best	Average	Worst
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

---

6. SORTING ALGORITHMS (VERY IMPORTANT FOR CCEE)

Algorithm	Best	Average	Worst
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$
Heap Sort	O(n log n)	O(n log n)	O(n log n)

📌 **CCEE TIP**:

- Best stable & guaranteed → **Merge Sort**
- Worst-case risk → **Quick Sort**

7. TREE OPERATIONS (Balanced Tree)

Operation	Time Complexity
Search	O(log n)
Insert	O(log n)
Delete	O(log n)
Traversal (all nodes)	O(n)

📌 Unbalanced tree → O(n)

---

8. GRAPH ALGORITHMS

Algorithm	Time Complexity
BFS	O(V + E)
DFS	O(V + E)
Dijkstra (array)	$O(V^2)$
Dijkstra (min-heap)	O((V + E) log V)

---

9. LOOP COMPLEXITY (MOST FREQUENT TRAPS)

Single Loop

for i = 1 to n

➞ O(n)

Nested Loop (Independent)

for i = 1 to n

 for j = 1 to n

➞ $O(n^2)$

Nested Loop (Dependent)

for i = 1 to n

for j = 1 to i

→ $O(n^2)$

Logarithmic Loop

for i = 1 to n

i = i * 2

→ O(log n)

Mixed Loop

for i = 1 to n

for j = 1 to log n

→ O(n log n)

---

10. RECURSION COMPLEXITY (HIGH WEIGHTAGE)

Recurrence Relation	Time Complexity
T(n) = T(n−1) + c	O(n)
T(n) = T(n/2) + c	O(log n)
T(n) = 2T(n/2) + n	O(n log n)
T(n) = 2T(n−1)	$O(2^n)$
Fibonacci recursion	$O(2^n)$

Stack Space:

- Linear recursion → O(n)
- Divide & conquer → O(log n)

---

11. COMMON PROGRAM PATTERNS

Pattern	Time Complexity
Factorial (iterative)	O(n)

Factorial (recursive)	$O(n)$
Fibonacci (recursive)	$O(2^n)$
Fibonacci (DP)	$O(n)$
Matrix traversal	$O(n^2)$
Subset generation	$O(2^n)$
Permutations	$O(n!)$

---

12. ASYMPTOTIC ORDER (MEMORIZE THIS)

$O(1)$

$< O(\log n)$

$< O(n)$

$< O(n \log n)$

$< O(n^2)$

$< O(2^n)$

$< O(n!)$
